AI Serving Framework — Project Prompt Pack

Seven production-shaped projects across GPU server, mobile and IoT

Project Prompt Pack

The failure these prompts are written against

Most AI projects do not fail because the model was wrong. They fail in four predictable ways, and every requirement in this pack exists to close one of them:

A demo that never becomes a system. Something works in a notebook, everyone is impressed, and nothing reaches a user. The gap is authentication, scaling, observability, rollout and rollback — none of which is interesting, all of which is the actual project.

A per-token bill that grows with success. Hosted APIs price the thing you were trying to increase. Costs scale with adoption, the model is deprecated on someone else's schedule, and the data leaves the building. Every project here self-hosts and states a fixed cost in the units the client actually pays.

Collapse under load, invisible on the dashboard. Inference saturates the accelerator, not the CPU, so a CPU-target autoscaler never fires and every infrastructure panel stays green while the product times out. Every project scales on the signal that genuinely saturates, and separates "the hardware is healthy" from "the system is useful" into two dashboards that are allowed to disagree.

Nobody can say what it was told, or who changed it. The prompt, the thresholds and the policies are the product's behaviour, and they usually live in a branch, a config file and someone's memory. A customer gets a wrong answer and no one can reconstruct what was live at the time. Every project puts that surface in front of a non-engineer, versions it immutably, and records every change in a tamper-evident log.

The frontend requirements are not decoration for the same reason. A buyer cannot evaluate a system they cannot see, and an internal team cannot align on one they cannot point at. The Architecture tab exists so that "why does this cost what it costs" has a visual answer, and the guided tour exists so the same answer is given the same way twice.

Copy-paste briefs for building production-shaped AI serving projects against seven different runtimes. Each is written to be pasted whole into a fresh repository as the opening specification.

The vLLM customer support assistant in this repository is the reference implementation. Where a prompt is ambiguous, that codebase is the tie-breaker — it is the worked example of the standard the others are held to.

How to use these

1. Create a new, empty repository for the project.
2. Paste `00-shared-requirements.md` and the project prompt together as the brief.
3. Feed **§0, the numbered build plan**, into Claude Code's plan mode. Every project prompt opens with one: twelve numbered steps, each independently reviewable, with the first four being what has to land before the demo is filmable.
4. Expect to answer the same four questions each project needs settled early: cloud target, compression target, scope of the configuration surface, and what hardware is available for development.

The prompts deliberately state acceptance criteria rather than implementation detail. They describe what must be true when it is finished, not how to type it.

What each project prompt now contains

Beyond the brief and the framework rationale, every prompt carries three sections that make the UI reproducible rather than merely described:

- **Architecture graph: node inventory** — the actual stage cards for that domain, in pipeline order, with what each expands into. This is the content for the 3D graph; the *mechanics* live in `00-shared-requirements.md` §2.
- **The improvement loop for this project** — which feedback signals are cheap to collect in that modality, and the domain rules that are expensive to get wrong (never train on captured faces; judge voice on the transcript; store the tile, not the frame).
- **The release gate for this project** — the two benchmark axes and the specific thresholds that matter, including where `lm_eval` and `GuideLLM` do not apply and what replaces them.

The shared requirements are unusually prescriptive about the graph

`00-shared-requirements.md` §2 reads as a long list of rules because each one was learned by getting it wrong on camera first: pinning two axes makes it look 2D, expanding in place buries the thing you are talking about, perspective-scaled icons are nine pixels wide, and re-framing the camera on every simulation settle makes dragging look broken. Follow it rather than re-deriving it.

The pack

#	Framework	Class	Project
—	**vLLM**	GPU server	Customer support assistant (**built — this repo**)
01	**TensorRT-LLM**	GPU server	Interactive voice agent, sub-800 ms turn latency
02	**SGLang**	GPU server	Document and image analysis, structured extraction
03	**llama.cpp**	Mobile	Offline field assistant, fully on-device
04	**ExecuTorch**	Mobile	On-device face verification, consented 1:1
05	**ONNX Runtime**	IoT / edge	Workplace safety tracking, zone intrusion
06	**LiteRT**	IoT / edge	Drone infrastructure inspection

Each project was matched to the framework whose actual strength it exercises, rather than distributed arbitrarily:

- **TensorRT-LLM → voice.** Time to first token sits directly in the turn-latency budget, and in-flight batching plus request cancellation are what make barge-in work. This is the workload where TensorRT-LLM's compilation cost is repaid.
- **SGLang → documents.** RadixAttention reuses the KV cache across branching shared prefixes, which is precisely what thousands of same-type documents produce. Constrained decoding removes invalid JSON as a failure class outright.
- **llama.cpp → offline mobile.** Minimal dependencies, GGUF quantization and `mmap` loading are what let a model run at all on a phone with no network.
- **ExecuTorch → face verification.** Direct PyTorch export, NPU delegation, and a small runtime, for a fixed vision pipeline that may need retraining on customer data.
- **ONNX Runtime → edge tracking.** One `.onnx` artifact across a heterogeneous device fleet via execution providers, which is the real operational problem at edge scale.
- **LiteRT → drone.** Best inference-per-watt on EdgeTPU-class hardware, plus pruning and quantization-aware training in one toolkit, for a workload where the compute budget is capped

by battery physics.

What every prompt enforces

Set out in full in `00-shared-requirements.md`:

- **Three frontend tabs** — Product, Architecture, Monitoring
- **A 3D click-to-focus architecture graph** where every component explains what it does, why it exists, what it saves the client, and what the user feels
- **The full component chain** — auth, authorization, retrieval, skills, inference, guardrails, stateless logging, audit, monitoring, staging/production, orchestration
- **Compression with a quality gate that can fail the build**
- **Hash-chained audit logging** with HIPAA / SOC 2 / GDPR control mapping
- **Prometheus and Grafana** covering latency, throughput and 4xx
- **Measured numbers, never estimated ones**

One rule worth reading before you start

The shared requirements carry a substitution rule, because applying the vLLM checklist unmodified to a vision model produces nonsense:

> **KV cache, continuous batching and prefix caching exist only where there is autoregressive token generation.** They apply to TensorRT-LLM, SGLang and llama.cpp. They do not apply to ExecuTorch face verification, ONNX Runtime object tracking, or LiteRT drone imagery — those run one forward pass per input and have no KV state to cache.

For the vision projects the equivalent story is INT8 quantization, operator fusion, NPU delegation, input resolution tuning and pruning. Each of those prompts names its substitute explicitly and asks for it to be shown on the architecture node in the same position the KV cache would occupy. The same applies to Kubernetes: mobile and IoT projects use a model registry with staged rollout rings, which is the honest analogue of staging production, not a lesser version of it.

Being right is worth more than being uniform. A client with domain knowledge will notice a fabricated KV cache immediately, and it costs more credibility than the consistency was worth.

Building the PDFs

```
./scripts/build-prompt-pdfs.sh
```

Writes one PDF per prompt to `prompts/pdf/`, plus a combined `all-projects.pdf`. Requires `pandoc` and a LaTeX engine (`xelatex`).

Copy from the markdown, not the PDF. The PDFs are for reading and sending to people; the `.md` files are the source you paste into a new repository. LaTeX sets common ligatures, so text copied out of a PDF can contain fl (U+FB02) where you expect `fl` — harmless when read, quietly wrong when pasted into a spec.